

Lernziele:

- Vertiefung des Umgangs mit einfach verketteten Listen (vgl. Blatt 12)
- Verwendung des Schlüsselwortes `readonly`

Vorbemerkung

Wir machen weiter mit dem, was wir in Session 12 zum Thema verkettete Listen programmiert haben. Ihr könnt dazu euren eigenen Code von Session 12 verwenden oder den Code von der Webseite (zu finden unter Session 16) laden. Hier das Programm in verkürzter Form:

```
1 @ <stdio.h>
2 @ <stdlib.h>
3
4 struct Node
5 {
6     val: int;
7     next: -> Node;
8 };
9
10 fn prependToList(list: -> Node, val: int)
11 { /* ... */ }
12
13 fn printList(list: -> Node)
14 { /* ... */ }
15
16 fn printListRec(list: -> Node)
17 { /* ... */ }
18
19 fn printListReverse(list: -> Node)
20 { /* ... */ }
21
22 fn freeList(list: -> Node)
23 { /* ... */ }
24
25 fn main()
26 {
27     local list: -> Node = nullptr;
28
29     for (local i: int = 1; i <= 10; ++i) {
30         prependToList(&list, i);
31     }
32
33     printf("list:\n");
34     printList(list);
35
36     printf("list:\n");
37     printListRec(list);
38
39     printf("list in reverse order:\n");
40     printListRec(list);
41
42     freeList(list);
43 }
```

Aufgabe

Macht euch zur Erinnerung nochmals mit dem Code und der Datenstruktur vertraut. Idealerweise so, dass ihr alles eurem Nebensitzer erklären könnt.

Schritt 2: Motivation für das Schlüsselwort `readonly`

Wenn eine Funktion zur Ausgabe einer Liste aufgerufen wird, erwartet man, dass dabei die Liste nicht verändert wird. Aber nehmen wir an, dass durch ein Copy-&-Paste-Missgeschick die Funktion wie folgt verändert wurde:

```
fn printList(list: -> Node)
{
    for (; list; list = list->next) {
        printf("val = %d\n", list->val);
        list->val *= 2;
    }
}
```

Nachdem der Wert eines Listenknotens ausgegeben wurde, wird der Wert verdoppelt. Nehmt im Code die Änderung vor und testet das Programm. Dabei werdet ihr natürlich beobachten, dass sich die Liste jedes Mal verändert, wenn man `printList` verwendet.

Nun ändert die Funktion wie folgt ab:

```
fn printList(list: -> readonly Node)
{
    for (; list; list = list->next) {
        printf("val = %d\n", list->val);
        list->val *= 2;
    }
}
```

Der Parameter `list` ist nach wie vor ein Zeiger auf einen Listenknoten, neu ist jedoch, dass hier das Schlüsselwort `readonly` verwendet wird. Die Bedeutung ist in diesem Fall, dass in der Deklaration vereinbart wird, dass das, was am Ende des Zeigers liegt, nicht verändert werden darf. Übersetzt man nun das Programm, liefert der Compiler eine Fehlermeldung:

```
MCL:session16 lehn$ abc list_1.abc
list_1.abc:26.14-26.17: error: : assignment of read-only variable 'list->val'

        list->val *= 2;
            ^^^
list_1.abc:26.18-26.20: error: operator can not be applied to operand 'list->val' of type
'const int' and operand '2' of type 'i32'
```

Wieso ist es gut, dass wir einen Fehler bekommen? Natürlich sind Fehler schlecht, aber es ist besser, einen Fehler **beim Übersetzen** zu bekommen als **zur Laufzeit**. Mit dem Schlüsselwort `readonly` haben wir dokumentiert und dem Compiler mitgeteilt, dass diese Funktion den Inhalt eines Listenknotens (also das, was am Ende des Zeigers auf den Listenknoten liegt) nicht verändern darf. Der Compiler erkennt, dass dieses Versprechen in Zeile 26 gebrochen wird. Wir können uns jetzt dafür entscheiden, dass die Funktion doch Knoten ändern darf (indem man `readonly` entfernt) oder den Fehler (d.h. die unabsichtliche Veränderung des Knotens) beheben.

Schritt 3: Generelle Verwendung von `readonly`

Die Verwendung von `readonly` ist nicht auf Zeigertypen beschränkt, sondern kann für jeden Variablentyp verwendet werden. Hier ein Beispiel für eine Integer-Variable:

```
@ <stdio.h>

fn main()
{
    local a: readonly int;

    a = 42;
    printf("a = %d\n", a);
}
```

Hier wird vereinbart, dass die Variable `a` im Folgenden nicht mehr verändert werden darf. Übersetzt man das Programm, bekommt man eine entsprechende Fehlermeldung. Man sieht hier

auch, dass es wenig Sinn macht, eine lokale `readonly` Variable bei der Deklaration nicht zu initialisieren. Man kann den Wert anschließend nicht setzen, und da es sich um eine lokale Variable handelt, hat diese stets einen undefinierten Wert.

In manchen Fällen soll die Initialisierung einer `readonly` Variable aber vom Wert einer anderen Variablen abhängen. Hier ein Beispiel, bei dem der Benutzer den Wert einer Variable `n` eingeben kann (Zeile 6). Ist der Wert von `n` größer als 10, soll `a` mit 42 und sonst mit 13 initialisiert werden (Zeile 9 bis Zeile 13):

```
1 @ <stdio.h>
2
3 fn main()
4 {
5     local n: int;
6     scanf("%d", &n);
7
8     local a: readonly int;
9     if (n > 10) {
10         a = 42;
11     } else {
12         a = 13;
13     }
14
15     printf("a = %d\n", a);
16 }
```

Offensichtlich kann dies nicht übersetzt werden, da versucht wird, den Wert von `a` nachträglich zu setzen. Man kann den Wert von `a` aber bei der Initialisierung in Abhängigkeit einer Bedingung wie folgt initialisieren:

```
1 @ <stdio.h>
2
3 fn main()
4 {
5     local n: int;
6     scanf("%d", &n);
7
8     local a: readonly int = n > 10 ? 42 : 13;
9
10     printf("a = %d\n", a);
11 }
```

In Zeile 8 wird hier auf der rechten Seite des Gleichheitszeichens eine sogenannte Conditional-Expression verwendet. Ist die Bedingung `n > 10` erfüllt, dann ist der Wert der Expression hier 42 (und allgemein das, was nach dem Fragezeichen kommt) und sonst 13 (und allgemein das, was nach dem Doppelpunkt kommt). Testet das Programm.

In ABC kann man für Conditional-Expressions eine alternative Notation verwenden, bei der man `?` mit `then` und `:` mit `else` ersetzt. Ändert Zeile 8 in:

```
local a: readonly int = n > 10 then 42 else 13;
```

und testet das Programm erneut.

Schritt 4: Zurück zur Liste

Aufgabe: Nach diesem Exkurs ändert das Programm für die Liste so, dass alle Funktionen zur Ausgabe der Liste einen Zeiger vom Typ

-> `readonly Node`

Node als Parameter erhalten.

Schritt 5: Knoten suchen

Aufgabe: Schreibt eine Funktion `isInList`, mit der man prüfen kann, ob in der Liste ein bestimmter Wert enthalten ist. Die Funktion soll `true` zurückgeben, falls dies der Fall ist, und ansonsten `false`. In der Funktion `main` soll man die Funktion wie folgt verwenden können:

```
printf("5 is %s list\n", isInList(list, 5) ? "in" : "not in");
```

Schritt 6: Knoten anhängen (auf die unschöne und ineffiziente Art)

Bisher können wir nur Knoten am Anfang der Liste einfügen. Für einfach verkettete Listen ist das auch die effizienteste (und einfachste) Methode, um neue Knoten hinzuzufügen. Aber es ist auch möglich, dass man am Ende der Liste einen Knoten anhängt. Eine Funktion `appendToList` kann man so implementieren, dass diese in der Funktion `main` analog zu `prependToList` verwendet werden kann:

```
83     local list: -> Node = nullptr;
84
85     for (local i: int = 1; i <= 10; ++i) {
86         appendToList(&list, i);
87     }
```

Die Implementierung ist aber im Vergleich zu `prependToList` deutlich komplizierter. Möglich wäre zum Beispiel diese Implementierung:

```
22 fn appendToList(list: ->-> Node, val: int)
23 {
24     local n: -> Node = malloc(sizeof(*n));
25     if (!n) {
26         printf("out of memory\n");
27         exit(1);
28     }
29     n->val = val;
30     n->next = nullptr;
31     if (*list) {
32         for (; (*list)->next; list = &(*list)->next) {
33             ;
34         }
35     } else {
36         *list = n;
37     }
38 }
```

In den Zeilen 24 bis 28 wird wie bisher Speicher für den Knoten allokiert. Der Wert des Knotens wird in Zeile 29 gesetzt, und in Zeile 30 wird festgelegt, dass der Knoten keine Nachfolger hat (da dieser ans Ende angefügt wird). Was die Sache nun kompliziert macht, ist, dass man unterscheiden muss, ob man den neuen Knoten an eine nicht-leere Liste anhängt oder in eine leere Liste einfügt. Bei einer leeren Liste ist `*list` der Null-Zeiger und die Bedingung in Zeile 31 `false`.

1. In den Zeilen 32 bis 34 wird der Knoten an das Ende einer nicht-leeren Liste angehängt. Dazu wird mit einer For-Schleife ans Ende der Liste iteriert. Dass hier `list` ein Zeiger auf einen Zeiger auf einen Knoten ist, macht die Sache dabei nicht einfacher. Nach der For-Schleife ist aber `*list` der Zeiger auf den letzten Knoten und an diesen wird der neue Knoten angehängt.
2. In Zeile 36 wird der Fall behandelt, dass die Liste leer ist. In dem Fall wird der Knoten wie in `prependToList` eingefügt.

Um das Anhängen eines Knotens in eine Liste zu unterstützen, lohnt es sich, die Datenstruktur anzupassen, um die Implementierung zu vereinfachen. Aber nicht nur das, bei obiger Implementierung hängt die Laufzeit davon ab, wie lange die Liste ist. Je länger die Liste ist, umso länger benötigt die Operation. Also da gibt's noch viel zu verbessern!!

Nichtsdestotrotz, testet die Implementierung und übt das Verständnis für Zeiger und Listen, indem ihr versucht, die Implementierung nachzuvollziehen.